

Lecture 11

this pointer

Today's Lecture Contents

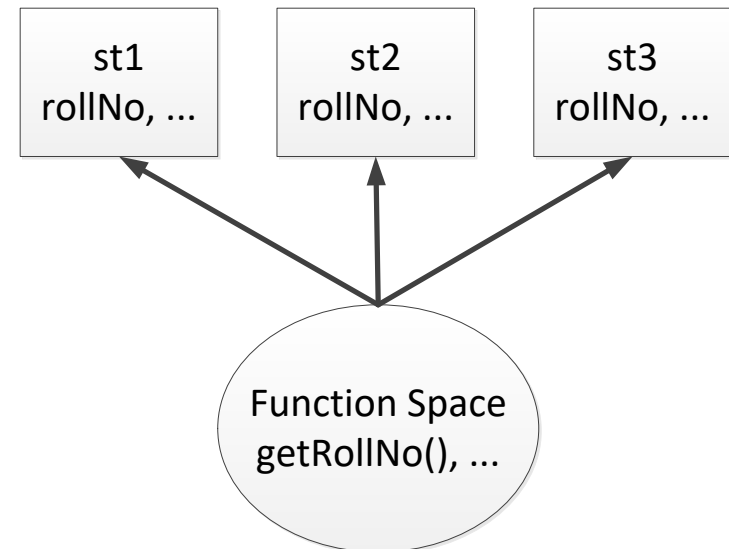
- This pointer

How does a member function know the object that called it?

- Each object gets its own copy of each of the **non-static data members** of the class.
 - These data members are sometimes referred to as instance variables since they are a property of each instance / object.
- All objects of a class share a single copy of each **static data member** of the class.
 - These data members are sometimes referred to as class variables since they are a property of the class as a whole rather than a particular object of the class.
- A single copy of each **member function definition** is stored in the code segment of the C++ program.
 - how does the code in that member function access the non-static data members for the correct object?

Member functions shared by all objects

- Suppose there is a class named Student
- When we create three objects of it
`Student st1, st2, st3;`
- Member function space is common for every object of a class
- Whenever a new object is created:
 - Memory is reserved for data members only
 - Previously defined member functions are used over and over again



this pointer

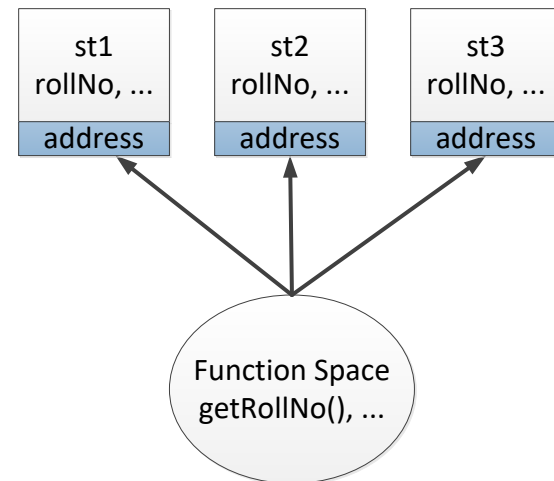
- **'this'** is a special kind of pointer.
- It is a pointer to the current instance of a class. It is used to refer to the object within its own member functions.
- When a member function is called, how does it know on which object to act on?
 - The compiler supplies a special implicit pointer named **this** that allows a member function to access the non-static data members.
 - The **this** pointer contains the address of the object that called the member function.
 - The **this** pointer is passed as an extra, hidden argument whenever a non-static member function is called and is available as a local variable within the body of all non-static member functions.
 - The **this** pointer is not available in static member functions since they can be called using a class name rather than the name of a specific object.

How **this** pointer is passed to the function?

- The **this** pointer is passed as an extra, hidden argument whenever a non-static member function is called and is available as a local variable within the body of all non-static member functions.
- '**This**' pointer contains the 'self-address' of the object. This address is dereferenced by the functions and hence they act on correct objects
- For example,

```
student s1("Ali");  
student s2("Ahmed");
```

```
s1.setID(1); //Passes &s1 as the this pointer for setID().  
s2.setID(2); //Passes &s2 as the this pointer for setID().
```



Using the this Pointer inside member function

- Most uses of the this pointer inside a member function's body will be implicit. Such as:

```
void student::setID(int i) {  
    ID = i;  
}
```

- We can also explicitly use `this` pointer:

```
void student::setID(int i) {  
    this->ID = i;  
    //OR  
    (*this).ID = i;  
}
```

Conversion during function invocation

- In the code, when we call

```
s1.setID(50);
```

- we are actually calling a function converted by compiler:

```
s1.setID(&s1, 50);
```

- The function itself should be converted from:

```
void setID(int i) {  
    ID = i;  
}
```

- to

```
void setID(student* const this, int i) {  
    this->ID = i;  
}
```


Implicitly and explicitly using this pointer

```
class Student {
string name;
int rollNo;
int age;
public:
Student(string aName="", int aNo=1,int
myage=20):name(aName),rollNo(aNo),age(myage) {}
void print() {
    //implicitly use this pointer
    cout<<"Student age is:  "<<age<<endl;
    //explicitly use dereferenced this pointer
    cout<<(*this).age<<endl;
    //explicitly use this pointer with arrow operator.
    cout<<this->age<<endl;
}
};

int main() {
Student s1;
    s1.print();
    return 0;
}
```

Using the this Pointer inside member function

- If you have a local variable in a member function that has the same name as a data member of the class, using the `this` pointer explicitly will allow the compiler to tell them apart: Such as:

```
void student::setID(int ID) {  
    this->ID = ID;  
}
```

-

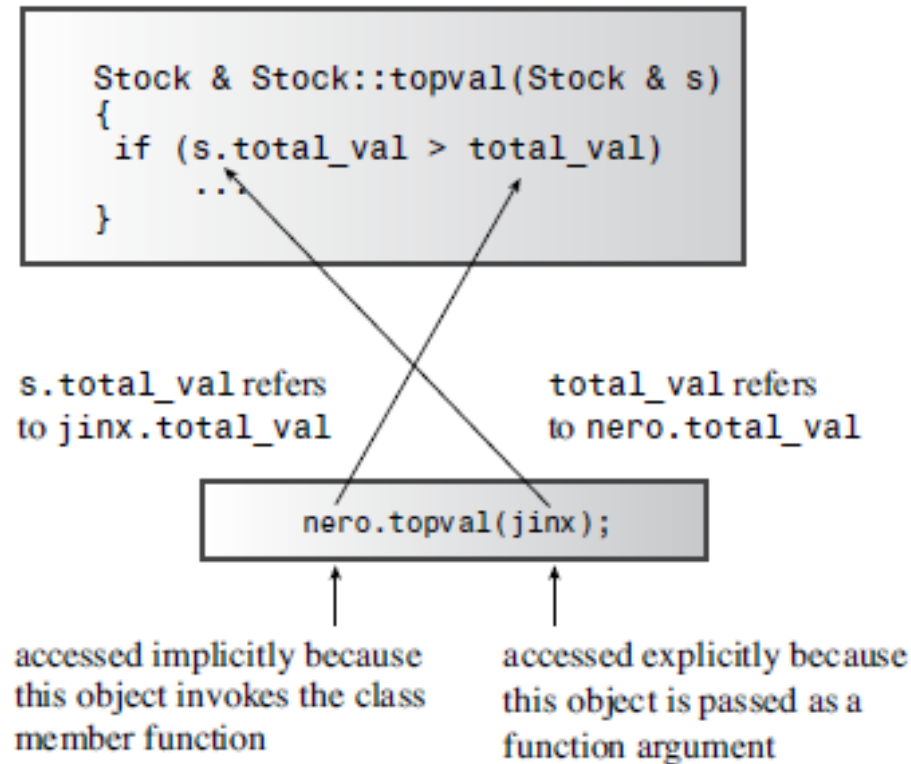
Another Example: Comparing two Stock objects

- Suppose we want a program that investigate a series of stocks to find the one with the greatest value

```
class Stock {
string company;
int shares;
double share_val, total_val;
void set_tot() { total_val = shares * share_val; }
public:
Stock(string co, long n, double pr) {
company = co; shares = n; share_val = pr; set_tot(); }
void show() const {
cout << "Company: " << company << " Shares: " << shares << '\n';
cout << "Share Price: $" << share_val;
cout << " Total Worth: $" << total_val << '\n'; }
Stock & topval(const Stock & s) const {
//compare total_val of two objects and return the greater one.
}
};
```

Example: Comparing two Stock objects

- Suppose we want a program that investigate a series of stocks to find the one with the greatest value



```
int main() {
    Stock nero("woof inc", Inc.", 10, 30);
    Stock jinx("pryal Co.", 10, 50);
    nero.topval(jinx);
}
```

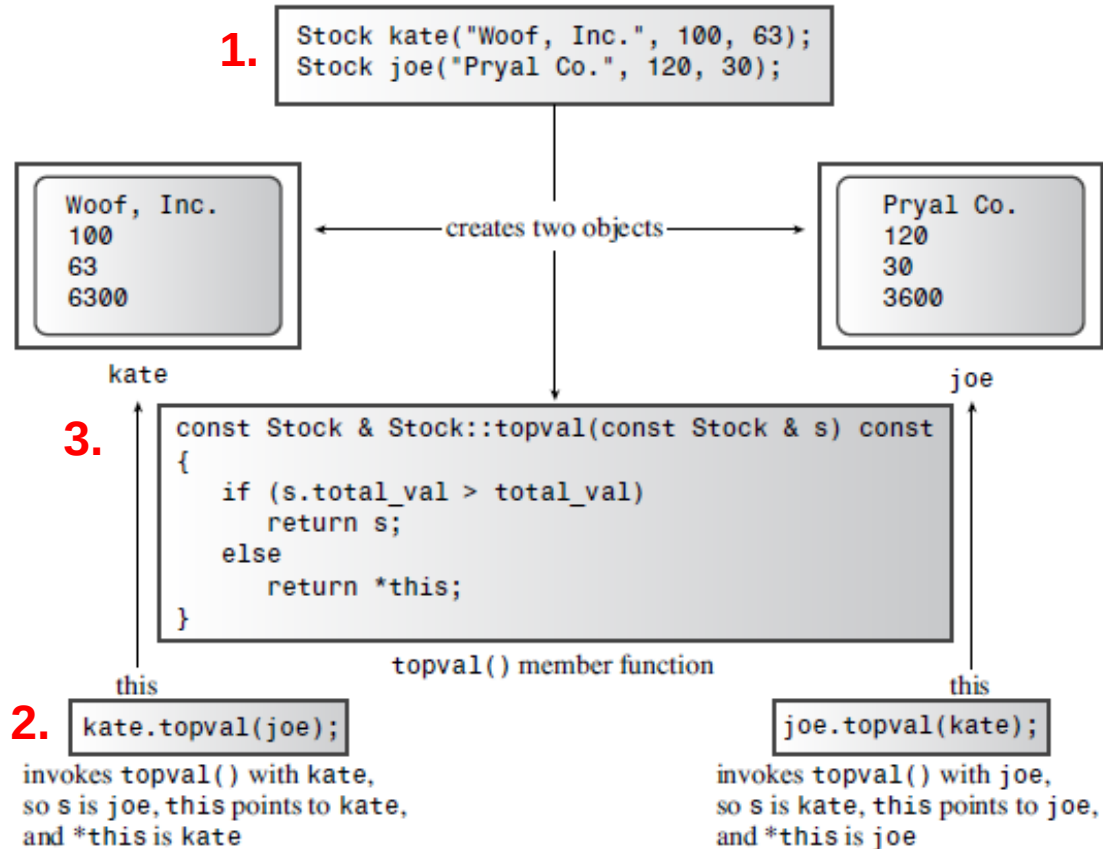
Knowing Your Objects: The **this** Pointer

```
class Stock {
string company;
int shares;
double share_val, total_val;
void set_tot() { total_val = shares * share_val; }
public:
Stock(string co, long n, double pr) {
company = co; shares = n; share_val = pr; set_tot(); }
void show() const {
cout << "Company: " << company << " Shares: " << shares << '\n'; cout << "
Share Price: $" << share_val; cout << " Total Worth: $" << total_val <<
'\n'; }
Stock & topval(const Stock & s) const {
if (s.total_val > total_val)
return s;
else return ???; }
};

int main() {
Stock kate("woof inc", Inc.", 10, 30);
Stock joe("pryal Co.", 10, 50);
Stock & top = kate.topval(joe); }
```

Example: Comparing two Stock objects

- The C++ solution to this problem is to use a special pointer called `this`.



```
int main() {
    Stock kate("woof inc", Inc.", 10, 30);
    Stock joe("pryal Co.", 10, 50);
    Stock &top = kate.topval(joe);}
```

Knowing Your Objects: The **this** Pointer

```
class Stock {
string company;
int shares;
double share_val, total_val;
void set_tot() { total_val = shares * share_val; }
public:
Stock(string co, long n, double pr) {
company = co; shares = n; share_val = pr; set_tot(); }
void show() const {
cout << "Company: " << company << " Shares: " << shares << '\n';
cout << "Share Price: $" << share_val;
cout << " Total Worth: $" << total_val << '\n'; }
Stock & topval(const Stock & s) const {
if (s.total_val > total_val)
return s;
else return *this; }
};

int main() {
Stock kate("woof inc", Inc.", 10, 30);
Stock joe("pryal Co.", 10, 50);
Stock &top = kate.topval(joe); }
```

Returning `this` Pointer for cascaded function calls

- There are situations where the programmer wants to return reference to current object from a function
- In such cases reference is taken from this pointer like (`*this`)
 - Pointer returned from the member function can be used for the cascaded function calls

```
classname & memfunc()  
{  
    //..body of the member function  
    return *this;  
}
```

- Cascaded member function calls
 - Other functions can operate on that pointer:
 - For example,

```
S.setName().setAge().setRollNo();
```


Returning `this` pointer (Example)

```
Student& Student::setRollNo(int aNo)
{
    ...
    return *this;
}

Student& Student::setName(char *aName)
{
    ...
    return *this;
}

int main()
{
    Student aStudent;
    aStudent.setName("John").setRollNo(1024);

    ...
    return 0;
}
```

Cascaded function calls using this pointer

```
class Student {
string name;
int rollNo;
int age;
public:
Student(string aName="", int aNo=1,int myage=20):
    name(aName),rollNo(aNo),age(myage) {}
Student &setRollNo(int aRollNo) {
    rollNo = aRollNo;
    return *this; //enables cascading;    }
Student &setage(int aAge) {
    age = aAge;
    return *this; //enables cascading;    }
Student &setName(string aName) {
    name = aName;          return *this; //enables cascading;    }
};

int main() {
    Student aStudent;
    aStudent.setName("Muhammad").setRollNo(1).setage(30);
    return 0;
}
```

using this pointer to enable cascading function calls

- The function that does not return anything should be called in the end of the list of the calls. For example,
 `S.setage().printage();`
- Function `setage()` returns this pointer, but `printage()` does not return pointer.

```
Student & setage(int a) { age = a; return *this; }
```

```
Void printage() { cout<<"student age:"<<age; }
```

Task: change the program such that it compares the two objects and prints the one with greater value

```
class Stock {
string company;
int shares;
double share_val, total_val;
void set_tot() { total_val = shares * share_val; }
public:
Stock(string co, long n, double pr) {
company = co; shares = n; share_val = pr; set_tot(); }
void show() const {
cout << "Company: " << company << " Shares: " << shares << '\n';
cout << " Share Price: $" << share_val;
cout << " Total Worth: $" << total_val << '\n'; }
Stock & topval(const Stock & s) const {
if (s.total_val > total_val)
return s;
else return *this; }
};
int main() {
Stock kate("woof inc", Inc.", 10, 30);
Stock joe("pryal Co.", 10, 50);
}
```